

Introduction

Tailux is a cutting-edge React-based dashboard theme designed for developers seeking efficiency, flexibility, and aesthetics. Built with Tailwind CSS, it offers a seamless blend of Dark & Light Layouts, ensuring a visually engaging user experience.

With its multi-color schemes, monochrome mode, and ultra-responsive layouts, Tailux adapts effortlessly to any project. Its modular architecture and lightweight dependencies promise fast performance and easy scalability.

Tailux comes loaded with powerful tools like React Hook Forms, TanStack Table, and Headless UI, paired with pre-built apps (Chat, Mail, Kanban, and more) and pages (Help, Auth, Errors, etc.) for faster development. Its creative widgets, advanced tables, and beautiful components make crafting interfaces a breeze.

Developers will appreciate the easy customization, Yarn build tools, and well-documented structure, while users enjoy stunning dashboards for diverse needs: Sales, Analytics, Crypto, LMS, and more.

Tailux combines functionality, performance, and design to deliver a theme that elevates your projects to the next level.

Prerequisite

Tailux is built using **ReactJS** and **Tailwind CSS**. To work effectively with Tailux, it's important to have a foundational understanding of these technologies. ReactJS enables you to build dynamic and interactive web applications, while Tailwind CSS allows for rapid styling using utility-first CSS classes.

In addition to ReactJS and Tailwind CSS, knowledge of HTML, CSS, and JavaScript is highly recommended. These core web development skills will help you understand and modify Tailux structure, design, and interactivity as needed.

For development, a reliable text editor or IDE, such as **Visual Studio Code** or **WebStorm**, is suggested for an enhanced coding experience.

To get started, ensure **Node.js** is installed on your machine. We recommend using the Node.js LTS version. You will also need a dependency manager like npm or yarn, with **yarn** being the preferred choice for Tailux.

To verify if Node.js is installed, run the following command:

```
node -v
```

To verify if yarn is installed, run the following command:

```
yarn -v
```

To ensure you are always using the latest features, bug fixes, and security updates, its important to stay up to date with the changelogs and new releases for the core technologies used in Tailux:

ReactJS: Check the React [blog](#) for updates on new features, improvements, and breaking changes.

Tailwind CSS: Follow the Tailwind [blog](#) for the latest updates to their utility classes, configurations, and plugins.

Tailux: Monitor the official Tailux [changelog](#) to stay informed about new features and best practices specific to this framework.

Keeping up with these updates will help you take full advantage of the latest enhancements and ensure your development process remains smooth and efficient.

Installation

Please download the latest version of Tailux from the official [ThemeForest](#) website and extract the zip file into your project directory. Once extracted, you will find two folders:

- **Demo**

This folder contains the demo pages of the Tailux theme. We do not recommend using this folder in your project, as it is intended for reference purposes only.

- **Starter**

The starter package contains all the necessary files to begin working with Tailux. Use this folder to build your application.

Navigate to the `starter` folder (where the `package.json` file is located) and run the following command to install the required dependencies:

```
yarn
```

This will create a `node_modules` folder in your project directory containing all the installed dependencies.

Layouts

Tailux provides two distinct layouts: `main-layout` and `sideblock`.

- **Main Layout**

The `main-layout` is the primary layout and is recommended for most applications. It offers a versatile and structured design, suitable for settings pages and prebuilt applications.

- **Main Sidebar**

Serves as the primary navigation hub.

- **Panel Sidebar**

Displays options or details depending on the selected item in the main sidebar.

- **Sideblock Layout**

The `sideblock` layout is designed for simpler applications, dashboards, and admin panels.

- **Sidebar**

Includes a single sidebar, making it ideal for use cases with streamlined navigation requirements.

For robust and feature-rich applications, the `main-layout` is the best choice due to its flexibility and comprehensive navigation system. The `sideblock` layout is well-suited for lightweight applications where simplicity and efficiency are paramount.

For changing the layout, you can update the `themeLayout` value in the `src/confings/theme.config.js` file.

```
export const defaultTheme = {  
  /// ...  
  themeLayout: "main-layout",  
  /// ...  
};
```

Development Server

After installing all the necessary dependencies, you can launch the development server by running the following command in your terminal:

```
yarn dev
```

Once the server is running, open your browser and navigate to <http://localhost:5173/>. The application will automatically reload whenever you make changes to the source files, providing a seamless development experience.

CSS

Tailux leverages the Tailwind CSS framework for styling its components, ensuring a modern and consistent design system. Additionally, it includes custom CSS classes tailored for specific components and third-party libraries.

All CSS classes are well-documented and designed for ease of use, making it simple to implement and customize styles. The custom styles are written using Tailwind `@apply` utility, providing a clean and maintainable approach to styling.

```

├─ styles
│   └─ app
│       └─ components          # Styling for core UI components
│           └─ ...
│       └─ forms              # Styling for form elements
│           └─ ...
│       └─ vendors            # Styling for third-party library integrations
│           └─ ...
│       └─ index.css          # Combines styles from components, forms, and vendors
│   └─ base.css               # Foundational theme styles
│   └─ index.css              # Primary CSS entry point
│   └─ layout.css             # Layout-specific styles (e.g., main-layout, sideblock)
│   └─ variants.css           # Styles for color themes and variants

```

Theme Config

The `src/configs/theme.config.js` file allows you to customize various aspects of the theme configuration, including:

- **themeMode**
Control the theme mode (e.g., light, dark, or system).
- **isMonochrome**
Enable or disable a monochrome mode
- **themeLayout**
Choose the layout type (e.g., main-layout or sideblock).
- **cardSkin**
Set the card appearance (e.g., bordered or shadow).
- **darkColorScheme**
Define the color scheme for dark mode.
- **lightColorScheme**
Define the color scheme for light mode.
- **primaryColorScheme**
Select the primary color palette.
- **defaultLang**
Specify the default language.

- **fallbackLang**

Set a fallback language.

- **direction**

Configure text direction (e.g., LTR or RTL).

- **notification**

Adjust notification settings, such as visibility and position.

```
import { colors } from "constants/colors.constant";

const DEFAULT_DARK_COLOR = "cinder";
// Options: "cinder", "navy", "mirage", "black", "mint"

const DEFAULT_LIGHT_COLOR = "slate";
// Options: "slate", "gray", "neutral"

const DEFAULT_PRIMARY_COLOR = "blue";
// Options: "indigo", "blue", "green", "amber", "purple", "rose"

// Default theme configuration
export const defaultTheme = {
  themeMode: "system",
  // Options: "light", "dark", "system"
  isMonochrome: false,
  // If true, the theme uses a monochrome color palette

  themeLayout: "main-layout",
  // Options: "main-layout", "sideblock"
  cardSkin: "bordered",
  // Options: "bordered", "shadow"

  // Dark mode color scheme
  darkColorScheme: {
    name: DEFAULT_DARK_COLOR,
    ...colors[DEFAULT_DARK_COLOR],
  },

  // Light mode color scheme
  lightColorScheme: {
    name: DEFAULT_LIGHT_COLOR,
    ...colors[DEFAULT_LIGHT_COLOR],
  },

  // Primary color scheme
  primaryColorScheme: {
    name: DEFAULT_PRIMARY_COLOR,
    ...colors[DEFAULT_PRIMARY_COLOR],
  },
}
```

```

    },

    // Language settings
    defaultLang: "en",
    fallbackLang: "en",
    direction: "ltr",

    // Notification settings
    notification: {
      isExpanded: false,
      position: "bottom-right",
      visibleToasts: 4,
    },
  },
};

```

Folder Structure

Tailux offers a clean and organized folder structure, making it easy to locate and refactor components and sections for your project. Its design supports seamless scalability for projects of any size.

```

├─ public
│   └─ images
│       └─ md
├─ src
│   └─ app
│       ├── contexts
│       │   └─ ...
│       ├── layouts
│       │   └─ ...
│       ├── navigation
│       │   └─ ...
│       ├── pages
│       │   └─ ...
│       ├── router
│       │   └─ ...
│       ├── assets
│       │   └─ ...
│       ├── components
│       │   ├── docs
│       │   │   └─ ...
│       │   ├── shared
│       │   │   └─ ...
│       │   ├── template
│       │   │   └─ ...
│       │   └─ ui
│       │       └─ ...

```

```
|   ├── configs
|   |   ├── auth.config.js
|   |   ├── breakpoints.config.js
|   |   └── theme.config.js
|   ├── constants
|   |   ├── app.constant.js
|   |   ├── colors.constant.js
|   |   └── countries.config.js
|   ├── hooks
|   |   └── ...
|   ├── i18n
|   |   ├── locales.constant.js
|   |   |   └── ...
|   |   ├── config.constant.js
|   |   └── langs.js
|   ├── middleware
|   |   ├── AuthGuard.jsx
|   |   └── GhostGuard.jsx
|   ├── styles
|   |   ├── app
|   |   |   ├── components
|   |   |   |   └── ...
|   |   |   ├── forms
|   |   |   |   └── ...
|   |   |   ├── vendors
|   |   |   |   └── ...
|   |   |   └── index.css
|   |   ├── base.css
|   |   ├── index.css
|   |   ├── layout.css
|   |   └── variants.css
|   ├── utils
|   |   └── ...
|   ├── App.jsx
|   └── main.jsx
├── tailwind_plugins
|   └── ...
├── .gitignore
├── .prettierignore
├── eslint.config.js
├── index.html
├── jsconfig.json
├── package.json
├── postcss.config.js
├── prettier.config.js
├── README.md
├── tailwind.config.js
├── vite.config.js
└── yarn.lock
```

Internationalization

Tailux supports multilingual applications and allows customization of internationalization through the `src/i18n/config.js` file. Translation files can be added or updated in the `src/i18n/locales/` directory. Additionally, the `src/i18n/langs.js` file should be modified to add or remove configurations for datepickers, date formats, flags, and translation files.

```
export const locales = {
  en: {
    label: "English",
    dayjs: () => import('dayjs/locale/en'),
    flatpickr: null,
    i18n: () => import("./locales/en/translations.json"),
    flag: 'united-kingdom'
  },
  // ...
}
```

To switch between languages, use the `useLocaleContext` hook in your components. Also you can use the `useTranslation` hook to access the translation strings.

Customization

Tailux has modular architecture that allows you to customize various configuration, including: theme, app, colors, layout, and more.

For customizing colors and theme and app constants refer to `src/constants` and `src/configs`

Starter

Steps to Add a New Page:

- **Define Your Page Name**

Choose a name for your page, example: `Example`

- **Create the Page File**

- Navigate to `src/app/pages/dashboards`
- Create your page folder and file, e.g., `example/index.jsx`.
- Inside the file, define the function with the same name as your page.

- **Add the Page to Navigation**

- Open `src/app/navigation/dashboards.js`.
- Add an object of navigation item for your page:
- **Add the Page to the Router**
 - Decide whether your page should be public, protected, or ghost.
 - Navigate to the relevant router file, e.g., `src/app/router/protected.jsx`.
 - Add the route for your page.

Congratulations! You are successfully added a new page. Feel free to explore and customize the page as needed.

Production Build

To prepare your project for production, follow these steps:

- **Build the Project:**

Use the following command to generate the production-ready build:

```
yarn build
```

- **Output Directory:**

The optimized production files will be generated in the `dist/` directory. This folder contains minified assets, ready to be deployed to a production server.

- **Preview the Build:**

To verify the production build locally, use the following command:

```
yarn preview
```

- **Deployment:**

Deploy the contents of the `dist/` folder to your preferred hosting provider, such as Vercel, Netlify, or any static hosting platform.